为 Deep Learning Club 学习教程做贡献零基础指南

面向第一次参与 Ulink 深度学习社讲义项目的零基础贡献指南，
涵盖 Git 流程、Markdown 与 MyST 写法。

项目仓库：ulink-deep-learning-club/ulink-deeplearningclub-handouts

在线文档：ulink-deep-learning-club.github.io/ulink-deeplearningclub-handouts

关键词：贡献指南　Git　GitHub　Markdown　MyST　深度学习

2026 年 11 月 2 日

目录

怎么用这份指南 **3**

1 项目里有什么 **4**

2 四个词：仓库、分支、提交、PR **7**

3 装好工具 **9**

4 最快的一次贡献：在网页上改一个错别字 **10**

5 完整流程：从 Issue 到合并 **11**

5.1 先开 Issue . . . 11
5.2 Fork . . . 12
5.3 Clone 到电脑 . . . 13
5.4 安装依赖并预览 . . . 14
5.5 新建分支 . . . 15
5.6 修改并检查 . . . 15
5.7 提交 . . . 15
5.8 推送并开 PR . . . 16
5.9 等检查、改问题、合并 . . . 17

6 Git 常用操作 **19**

6.1 改错了怎么办 . . . 19
6.2 遇到冲突 . . . 19

7 Markdown 基础 **20**

8 本项目的 MyST 写法 **21**

8.1 文件骨架 . . . 21
8.2 引用其他章节 . . . 22
8.3 提示框 . . . 22
8.4 Mermaid 图 . . . 22
8.5 数学公式 . . . 23
8.6 代码 . . . 23
8.7 表格 . . . 23
8.8 文献 . . . 24
8.9 目录树 . . . 24

9 写作要求 **25**

10 代码贡献 **26**

11 翻译贡献 **27**

12 提交前检查 27

13 常见问题 28

14 术语表 29

15 接下来做什么 30

16 相关链接 30

怎么用这份指南

你的情况	先读
先认识仓库页面	第 1 节
安装本地环境	第 3 节
只想改一个错别字	第 4 节
第一次完整走 PR 流程	第 3、5 节
要新增或改写章节	第 5、8、9 节
要改代码示例	第 5、8、10 节
要做英文翻译	第 5、11 节
遇到报错	第 13 节

1 项目里有什么

仓库页面顶部有一排标签，它们负责不同操作，不是目录：

标签	用途
Code	浏览文件、目录和提交历史。大部分时间在这里。
Issues	讨论问题和提出修改建议。
Pull requests	提交、审阅和合并修改。
Agents	GitHub 的自动化代理入口，本课程用不到。
Actions	查看自动构建和部署结果。
Projects	项目看板，本课程用不到。
Security and quality	安全扫描和依赖提醒，普通贡献者用不到。
Insights	贡献者、提交和访问统计。
Settings	仓库设置，只有维护者会动。

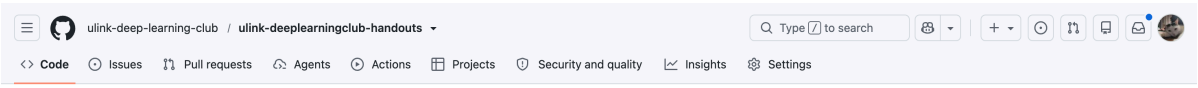


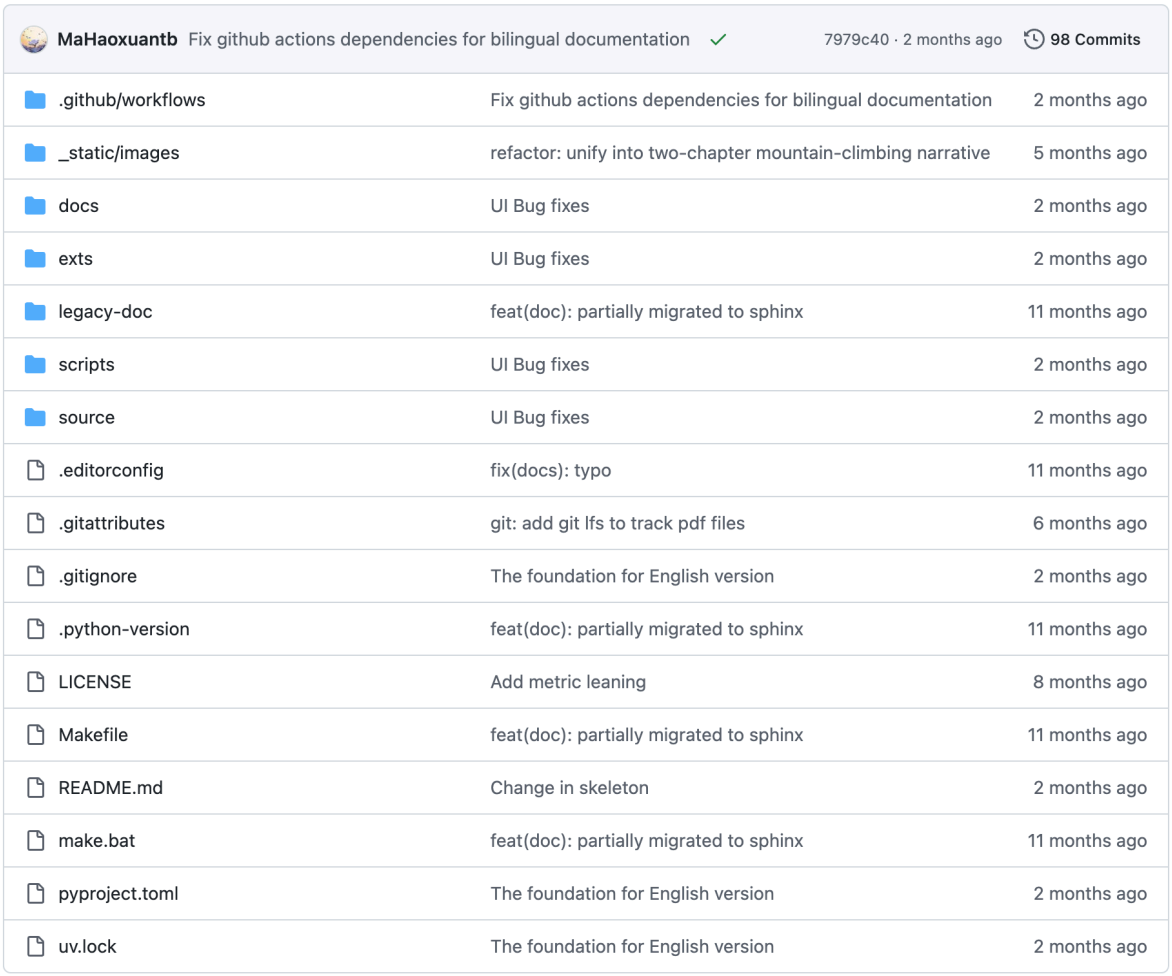
图 1: 仓库页顶部的标签栏。贡献时最常用的是 Code、Issues 和 Pull requests。

点 Code，页面下方显示当前分支的文件和目录：
这些路径在 Code 标签页里都能看到，也都位于项目仓库的 main 分支：

路径	内容	你需要动它吗
source/	课程正文，按章节分目录	会，大部分修改在这里
source/appendix/sphinx-guide/index.md	MyST 语法参考	写文档时查
source/references.bib	全局参考文献	加引用时动
source/locale/en/LC_MESSAGES/	英文翻译目录（.po）	做翻译时动
docs/	开发说明和本地化指南	查，不常改
scripts/	构建和中英文检查脚本	一般不动
legacy-doc/	已归档的 LaTeX 文档	不要动
build/、.venv/、cache/	编译产物和虚拟环境	不要提交

大部分改动集中在 source/。点进去会看到 math-fundamentals/、cnn-expedition/、pytorch-practice/ 等章节目录，每个目录里通常有 index.md、若干内容文件和 the-end.md。

正文源文件只有一份，就是 source/ 下的中文 Markdown。网页、PDF 和英文版都从它生成：



MaHaoxuantb Fix github actions dependencies for bilingual documentation ✓	7979c40 · 2 months ago	98 Commits
.github/workflows	Fix github actions dependencies for bilingual documentation	2 months ago
_static/images	refactor: unify into two-chapter mountain-climbing narrative	5 months ago
docs	UI Bug fixes	2 months ago
exts	UI Bug fixes	2 months ago
legacy-doc	feat(doc): partially migrated to sphinx	11 months ago
scripts	UI Bug fixes	2 months ago
source	UI Bug fixes	2 months ago
.editorconfig	fix(docs): typo	11 months ago
.gitattributes	git: add git lfs to track pdf files	6 months ago
.gitignore	The foundation for English version	2 months ago
.python-version	feat(doc): partially migrated to sphinx	11 months ago
LICENSE	Add metric leaning	8 months ago
Makefile	feat(doc): partially migrated to sphinx	11 months ago
README.md	Change in skeleton	2 months ago
make.bat	feat(doc): partially migrated to sphinx	11 months ago
pyproject.toml	The foundation for English version	2 months ago
uv.lock	The foundation for English version	2 months ago

图 2: Code 标签页下方的文件列表。目录名就是你在正文里引用的路径。

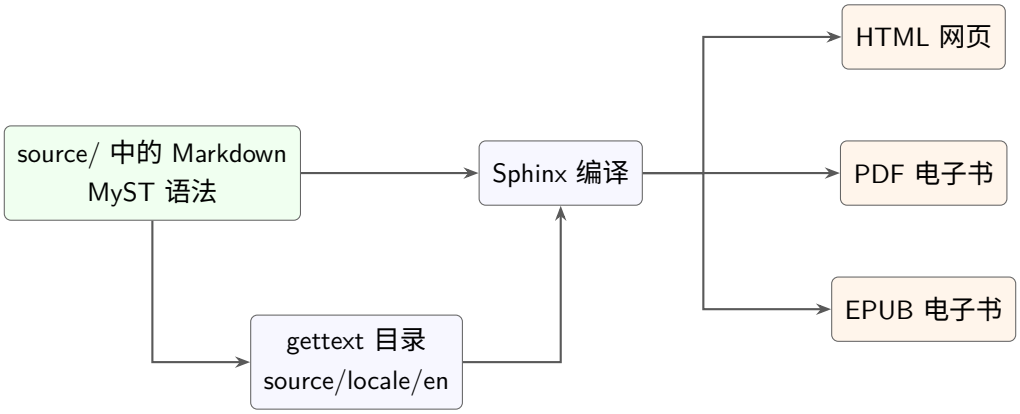


图 3: 唯一的内容源与三种输出格式

编译后的网页长这样：

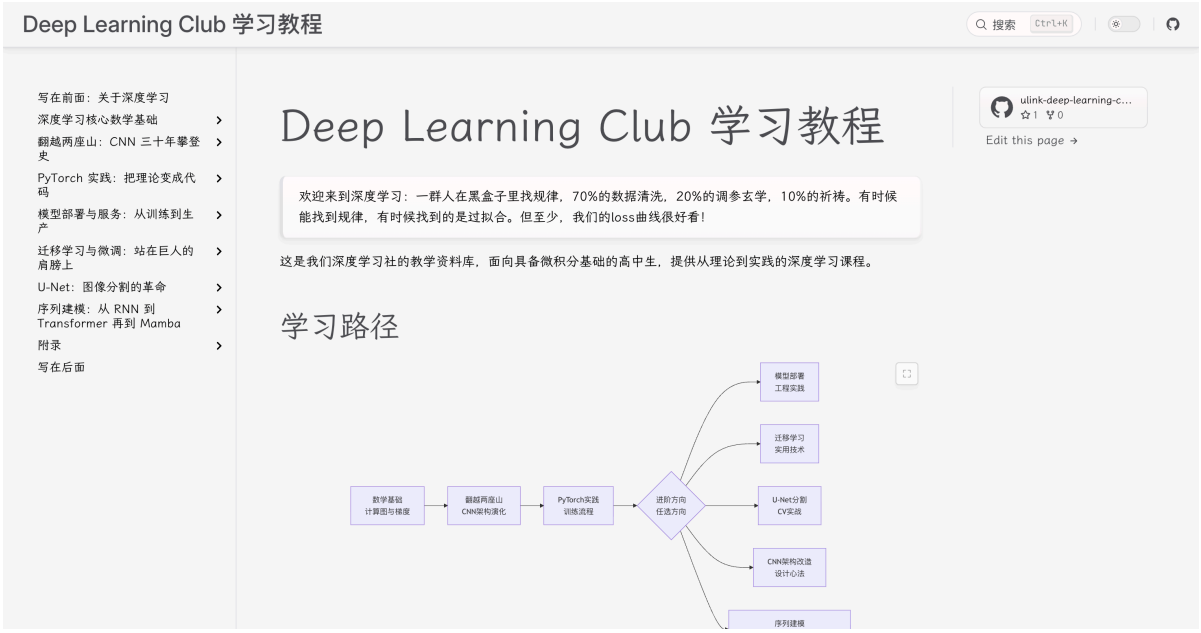


图 4: 编译后的在线文档。你改的是源文件，读者看到的是这个页面。

你改的是左边的源文件，读者看到的是编译后的页面。源文件里写错一个标签，编译时会出现警告，页面上的链接也可能断掉。

2　四个词：仓库、分支、提交、PR

Git 管的是文件的历史。你可以把它想成游戏存档：每次提交是一个存档点，分支是另一条存档线，合并就是把一条存档线接回主线。

你会接触两份仓库：

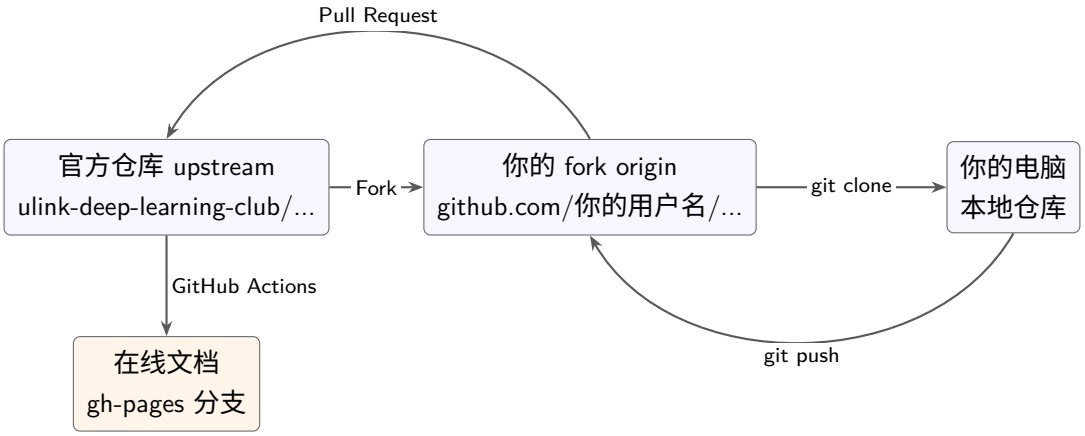


图 5: 官方仓库、你的 fork 与本地仓库的关系

词	意思	在本项目里对应什么
仓库 repository	一个带历史记录的项目目录	整个讲义
Fork	在 GitHub 上复制一份别人的仓库到你的账号	你的个人副本
分支 branch	仓库里一条独立的修改线	`fix/typo-preface` 这类名字
提交 commit	一次有说明的修改记录	改完一段话后的存档
远程 remote	另一份仓库的地址	`origin` 和 `upstream`
Pull Request (PR)	请求官方仓库合并你的分支	审核和合并的地方
Issue	讨论问题或提出修改建议的帖子	动手前先对齐方向

`origin` 指向你的 fork，你往这里推送。`upstream` 指向官方仓库，你从这里拉取最新内容。两个名字只是约定，但项目里的命令都按这个约定写。

文件从编辑到进入历史，会经过这几个位置：

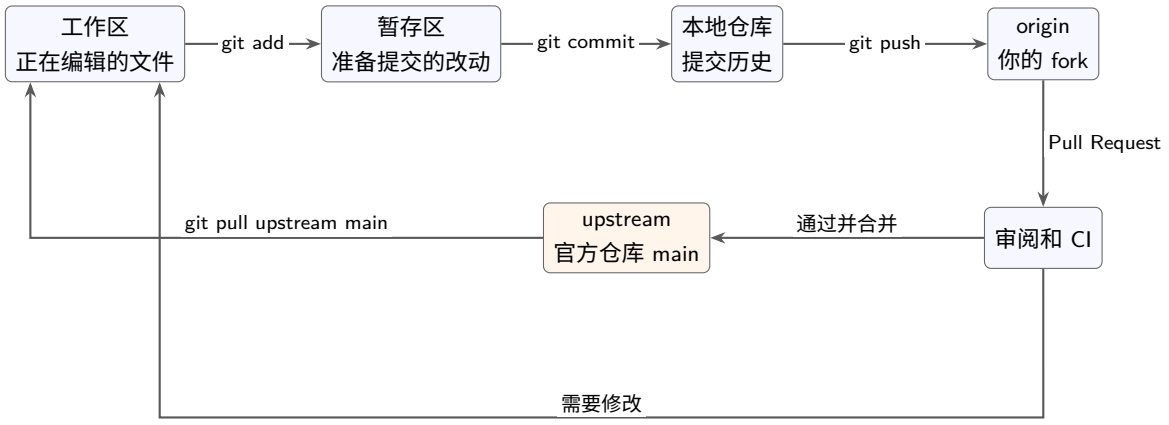


图 6: 一次修改从工作区到合并的位置变化

分支在历史里是这样分开又合回来的：

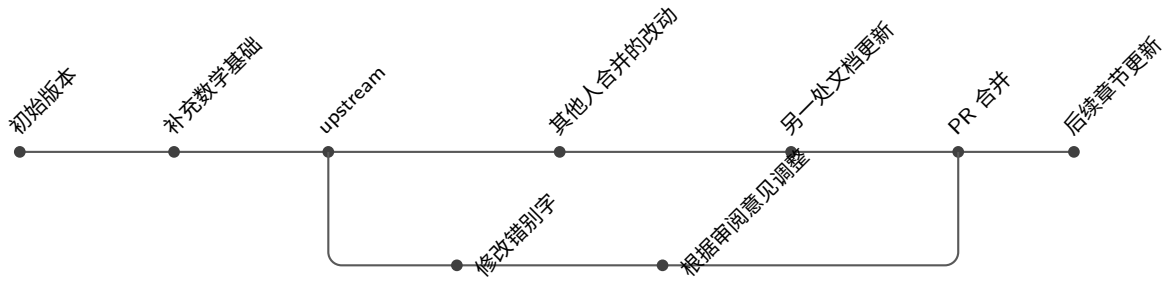


图 7: 分支的创建、提交与合并

GitHub 的分支页面可以查看当前有哪些分支：

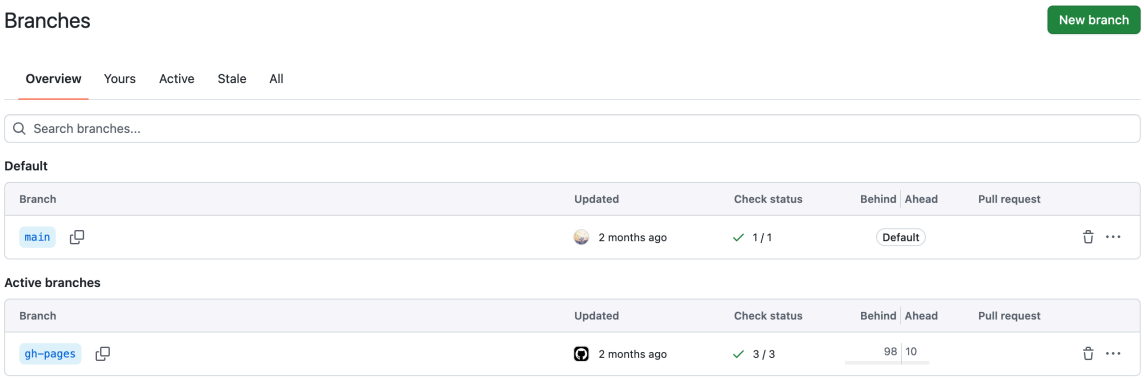


图 8: 分支页面。main 是默认分支，gh-pages 用来发布网页。

3 装好工具

最少需要 Git 和一个编辑器。构建网页还需要 Python 和 uv。画 Mermaid 图或生成 PDF 时，才需要 Node、Chromium 和 TeXLive。

工具	用途	检查命令
Git	保存和同步修改	`git --version`
GitHub 账号	托管 fork、Issue 和 PR	浏览器登录
编辑器	改 Markdown	VS Code、Zed、Cursor 都可以
Python 3.13+	运行 Sphinx，版本以 `.python-version` 为准	`python3 --version`
uv	安装 Python 依赖	`uv --version`
Node.js + mermaid-cli	渲染 Mermaid	`mmdc --version`
TeXLive + pdf2svg	生成 PDF 和 TikZ	`pdflatex --version`

Git 第一次使用要先写名字和邮箱：

```
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"
```

这些信息会写进提交记录。邮箱用你愿意公开的那个。

Tip

命令里的尖括号表示占位符，例如 <文件>。执行时换成真实内容，不要保留尖括号。

macOS 可以用 Homebrew 安装：

```
brew install git uv node
npm install -g @mermaid-js/mermaid-cli
```

Windows 安装 Git for Windows，在 Git Bash 里执行文中的 bash 命令。本文用命令行；你也可以用 GitHub Desktop 完成同样的操作，但排错时命令更容易描述。

uv 的其他安装方式见官方文档。

Ubuntu 和 Debian 的完整依赖在本地构建说明里。只改文字时不用装 TeXLive，`make html` 可以正常生成网页，缺少 Mermaid 或 TikZ 时对应图表不显示。

4　最快的一次贡献：在网页上改一个错别字

如果只改一句话，可以完全不装 Git。打开官方仓库里的目标文件，点右上角的铅笔图标：

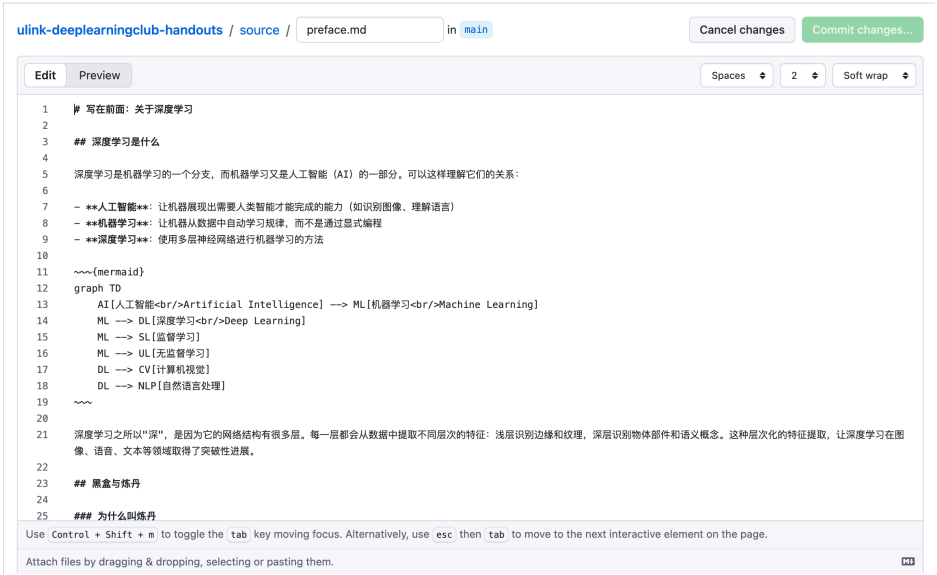


图 9: 网页编辑器。改完可以切到 Preview 看格式，再点 Commit changes。

步骤是这样：

1. 在文件页面点铅笔图标，GitHub 会自动帮你 fork 一份。
2. 在 `Edit` 里改文字，切到 `Preview` 看效果。
3. 点 `Commit changes`。
4. 选择 `Create a new branch for this commit and start a pull request`，给分支起名，例如 `fix/typo-preface`。
5. 点 `Propose changes`，填写 PR 说明，再点 `Create pull request`。

这条路径适合错别字、标点、一句话的语病。新增章节、改结构、加代码或做翻译时，用第 5 节的本地流程更方便，因为你能在提交前完整编译一次。

5 完整流程：从 Issue 到合并

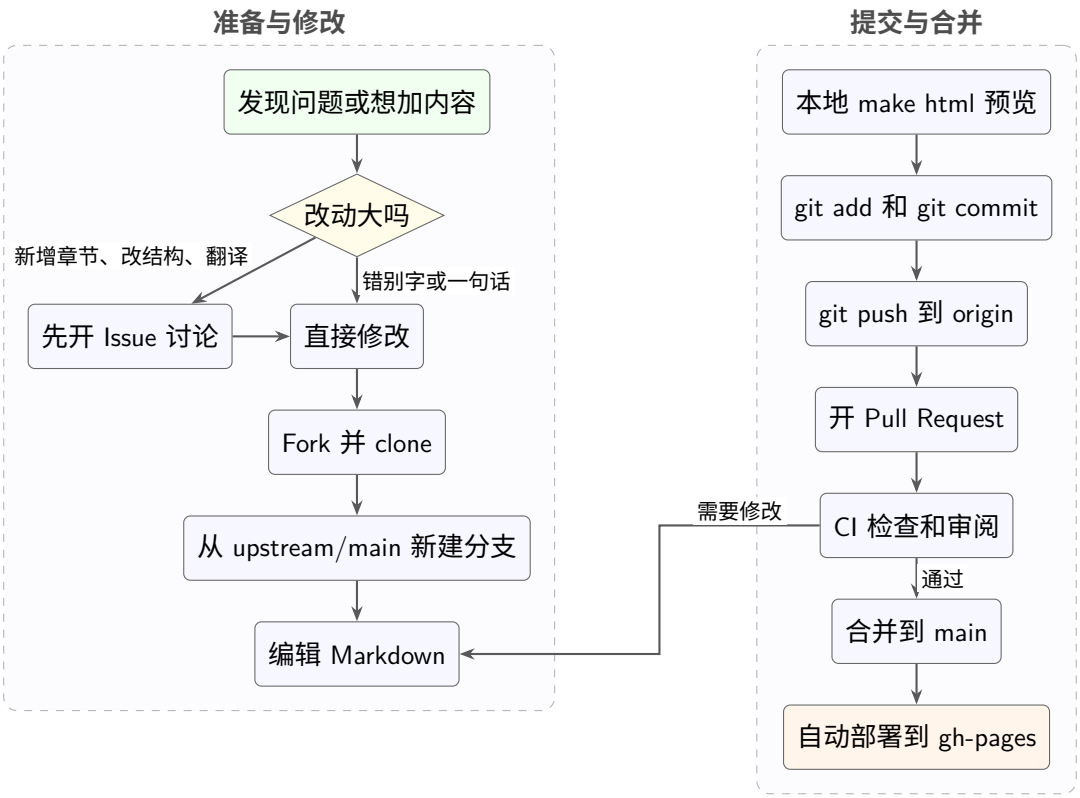


图 10: 从发现问题到部署上线的完整流程

5.1 先开 Issue

项目约定先讨论再动手。错别字可以直接提 PR，其他改动建议先开 Issue，尤其是新增章节、删除内容、调整目录结构和翻译。

打开 `Issues`，点 `New issue`：

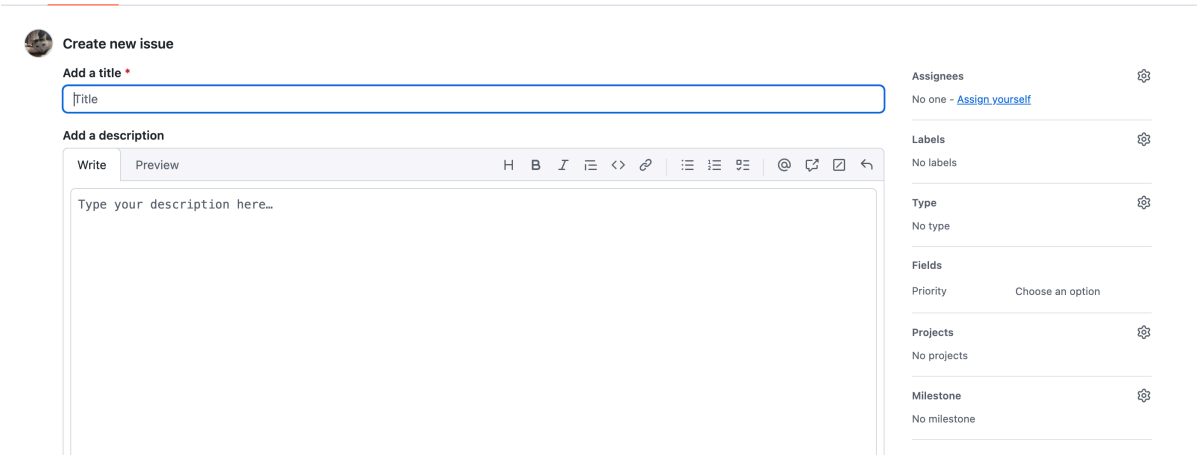


图 11: 新建 Issue。标题写类型和位置，正文写现状、建议和原因。

标题写清楚类型和位置，例如：

```
docs：前言中的“机器学系”应为“机器学习”
```

正文可以按这个格式：

```
## 位置

`source/preface.md` 第 12 行

## 现在的内容

机器学系是人工智能的一个分支。

## 建议改成

机器学习是人工智能的一个分支。

## 原因

“机器学系”是错别字，读者会困惑。
```

Warning

Issue 里不要放 API key、密码、个人身份信息或未公开的数据。需要贴截图时，先把无关的窗口和用户名遮掉。

5.2　Fork

打开官方仓库，点右上角的 Fork：

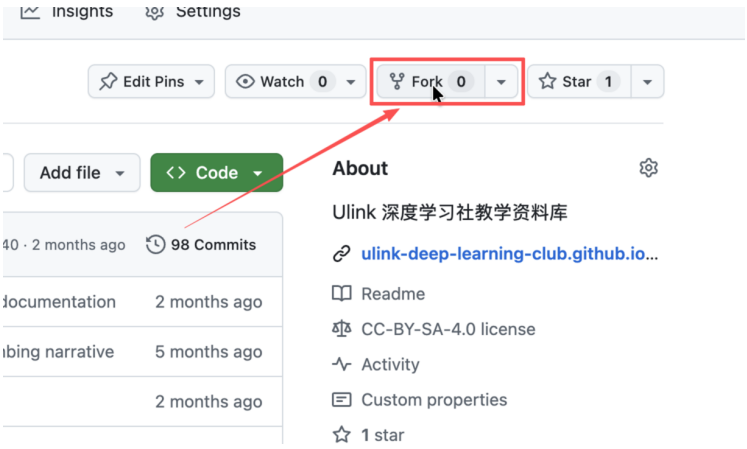


图 12：仓库右上角的 Fork 按钮。

GitHub 会打开创建 fork 的页面：

保持默认名字，点 Create fork。完成后，你的账号下会有一份 ulink-deeplearningclub-handouts。

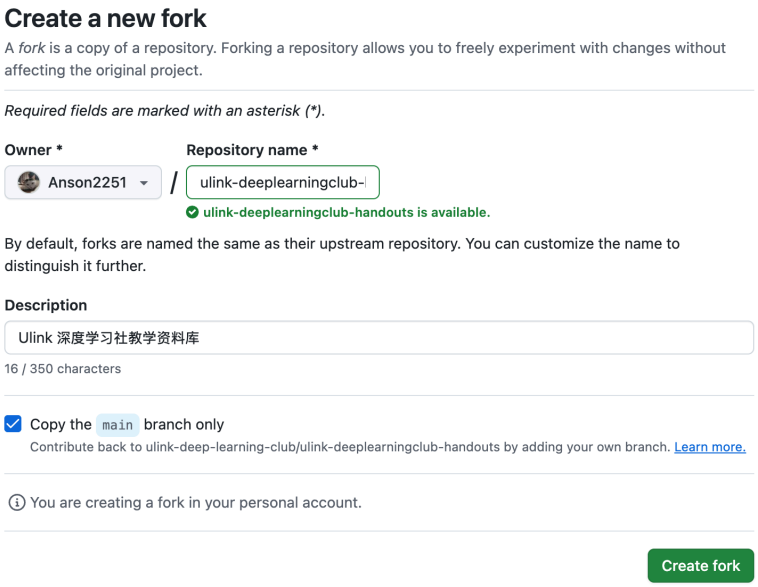


图 13: 创建 fork。Owner 选你的账号，名字保持默认，右下角点 Create fork。

5.3 Clone 到电脑

在你的 fork 页面点绿色的（示例中的 clone 菜单是官方仓库的，这里需要用你自己 fork 的仓库）`Code` 按钮，复制 HTTPS 地址：

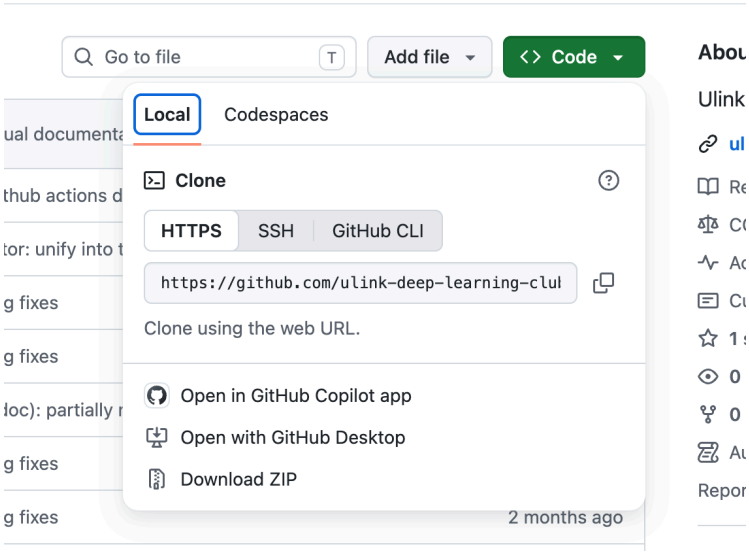


图 14: Code 菜单里的 HTTPS 地址，点右边的复制按钮。

然后在终端里执行下面的命令，把 **你的用户名** 换成你的 GitHub 用户名。

```
git clone https://github.com/你的用户名/ulink-deeplearningclub-handouts.git
cd ulink-deeplearningclub-handouts
```

把官方仓库添加为 upstream:

```
git remote add upstream https://github.com/ulink-deep-learning-club/ulink-
    deeplearningclub-handouts.git
git remote -v
```

git remote -v 应该显示四行，origin 两行指向你的 fork，upstream 两行指向官方仓库。如果提示 remote upstream already exists，说明已经加过了，不用重复加。地址不对时用:

```
git remote set-url upstream https://github.com/ulink-deep-learning-club/ulink-
    deeplearningclub-handouts.git
```

5.4 安装依赖并预览

所有构建命令都在仓库根目录执行:

```
uv venv
uv sync
source .venv/bin/activate
make html
```

Windows PowerShell 的激活命令是:

```
.venv\Scripts\activate
```

Windows 上在 PowerShell 或 cmd 里运行 make.bat html，Git Bash 里没有 make。如果 uv 不可用，可以退回 Python 自带的虚拟环境:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e .
make html
```

构建完成后启动本地服务器:

```
cd build/html
python -m http.server 8000 --bind 127.0.0.1
```

浏览器打开 http://localhost:8000。服务器会占用一个终端；改完源文件后，另开一个终端回到仓库根目录，再运行 make html，然后刷新浏览器就能看到变化。

如果你在 build/html 里迷路了，回到仓库根目录:

```
cd "$(git rev-parse --show-toplevel)"
```

只改中文正文时，make html 就够了。要检查中英文切换和 /en/ 页面，用：

```
bash scripts/build-i18n-site.sh --html-only
```

5.5 新建分支

不要直接在 main 上改，先从官方仓库拉最新内容，再开分支：

```
git fetch upstream
git switch -c fix/typo-preface upstream/main
```

分支名用能看懂的类型前缀：

前缀	用途	例子
fix/	修错误	fix/typo-preface
docs/	文档调整	docs/clarify-backprop
feat/	新增内容	feat/lstm-section
i18n/	翻译	i18n/translate-preface

5.6 修改并检查

用编辑器打开文件。在 GitHub 文件页面按 t 可以打开文件搜索，或者点仓库首页的 Go to file。在本地不知道文字在哪个文件时，用 git grep：

```
git grep -n "机器学系"
```

它会输出文件名、行号和匹配内容。改完先看状态和差异：

```
git status
git diff
```

git diff 里，- 开头是删掉的内容，+ 开头是新增的内容。提交前把整个 diff 读一遍，确认没有误删、没有把个人路径写进去、没有带进 build/ 或 .venv/。

5.7 提交

只添加你改过的文件：

```
git add source/preface.md
git commit -m "docs(preface): 修正“机器学系”错别字"
```

提交信息参考现有历史，格式是 类型(范围)：说明：

类型	用途	例子
`feat`	新增章节或功能	`feat(sequence-modeling): 增加 LSTM 一节`
`fix`	修正错误	`fix(cnn-expedition): 修正 ResNet 参数量`
`docs`	文档措辞或结构	`docs(preface): 补充鱼书对照说明`
`refactor`	调整结构但不改内容	`refactor(cnn-expedition): 合并两节`
`i18n`	翻译和本地化	`i18n(preface): 更新英文目录`

一次提交只做一件事。改错别字和调整目录不要放进同一个提交，审阅时很难看清。一个 PR 也只解决一个问题：格式调整、内容改写和翻译不要混在同一个 PR 里。

5.8 推送并开 PR

把分支推到你的 fork：

```
git push -u origin fix/typo-preface
```

推送后打开你的 fork，GitHub 会显示 `Compare & pull request`。点它，确认这几个字段：

字段	应该是什么
base repository	`ulink-deep-learning-club/ulink-deeplearningclub-handouts`
base branch	`main`
head repository	你的 fork
compare branch	你刚推的分支，例如 `fix/typo-preface`

PR 页面长这样（这是一个已经合并的 PR 示例，重点看状态和标签页）：

标题写清楚改动，正文用这个模板：

```
## 改了什么

把 `source/preface.md` 第 12 行的“机器学系”改成“机器学习”。

## 为什么

原文是错别字，会影响读者理解。

## 怎么验证

- [x] 本地运行 `make html`
- [x] 打开 `build/html/preface.html` 检查文字
- [x] 没有新增构建警告

Closes #12
```

最后一行 `Closes #12` 会在 PR 合并后自动关闭对应的 Issue。没有 Issue 就删掉这行。

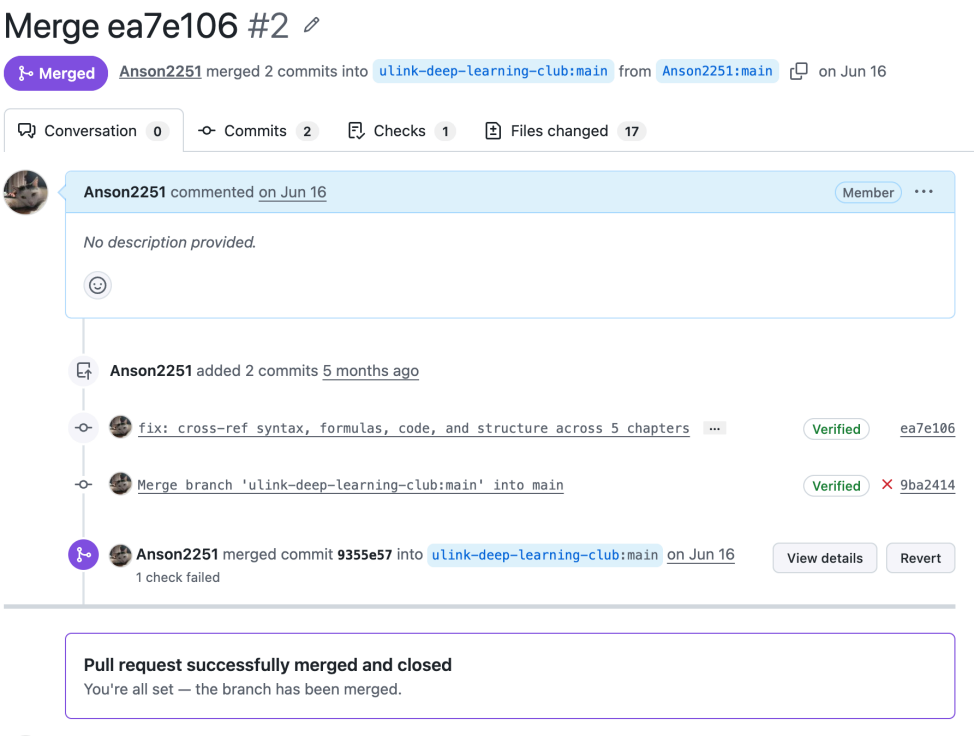


图 15: PR 页面。Conversation 看讨论，Commits 看提交，Checks 看构建，Files changed 看 diff。

Tip

如果还没改完，可以先点 `Create draft pull request`，改完再点 `Ready for review`。

5.9 等检查、改问题、合并

PR 打开后，GitHub Actions 会自动构建。绿色的勾表示通过：

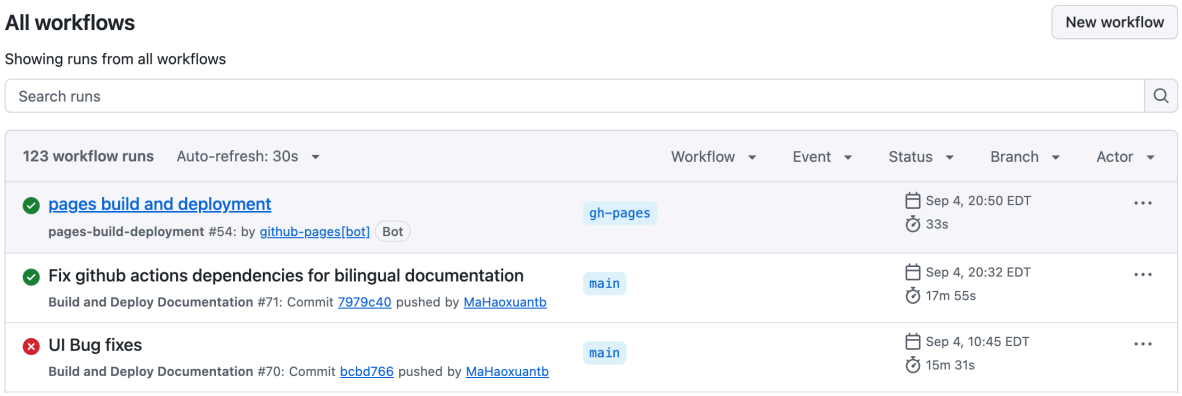


图 16: Actions 页面。绿色对勾表示构建通过，红色叉号表示失败。

点 Checks 可以看日志。构建失败时，先找第一个 `ERROR` 或 `WARNING`，那通常才是原因。审阅者提了修改意见时，不需要新开 PR。在同一个分支继续改，然后：

```
git add <改过的文件>
git commit -m "docs(preface): 根据审阅意见调整措辞"
git push
```

PR 会自动更新。`Files changed` 页面可以看完整 diff：

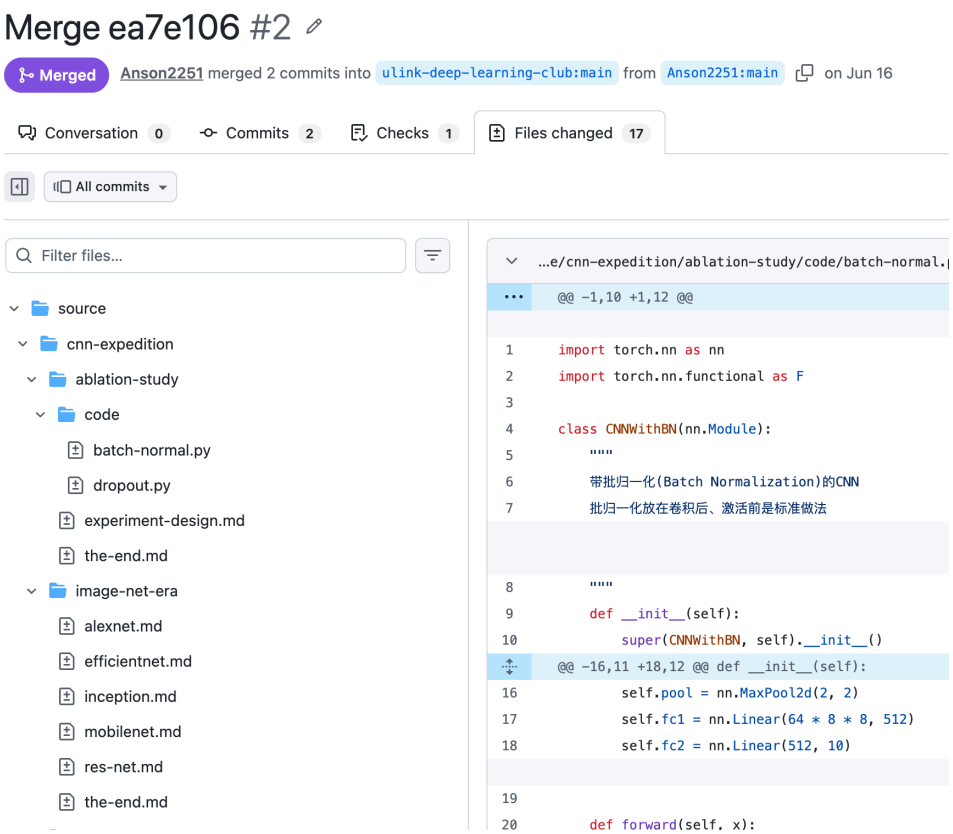


图 17: Files changed 页面。左边选文件，右边看逐行改动。

审阅者通过后，维护者会点 `Merge pull request`。合并完成后可以删除远程分支，GitHub 页面上有按钮。

回到本地，把 `main` 更新到最新：

```
git switch main
git pull --ff-only upstream main
git push origin main
```

如果你的 fork 页面显示 `This branch is behind`，也可以点 `Sync fork`。

6 Git 常用操作

目的	命令
看当前状态	`git status`
看未暂存的改动	`git diff`
看已暂存的改动	`git diff --staged`
暂存一个文件	`git add <文件>`
提交	`git commit -m "说明"`
推到 fork	`git push`
拉官方最新 main	`git pull --ff-only upstream main`
切换分支	`git switch <分支名>`
新建并切换分支	`git switch -c <分支名> upstream/main`
看提交历史	`git log --oneline --graph`
撤销已推送的提交	`git revert <commit>`
临时保存未提交的改动	`git stash`
取回临时保存的改动	`git stash pop`

6.1 改错了怎么办

还没 `git add`:

```
git restore <文件>
```

已经 `git add`，但还没提交:

```
git restore --staged <文件>
```

已经提交，但还没推送:

```
git reset --soft HEAD~1
```

这会把最后一次提交撤回暂存区，文件内容还在。不要用 `git reset --hard`，它会丢掉最后一次提交和未提交的修改。

已经推送的提交不要改写历史。要撤销就运行 `git revert <commit>`，它会生成一个反向提交；也可以直接再做一次修正提交。共享分支上执行 `git push --force` 会影响别人的工作。

6.2 遇到冲突

冲突时 Git 会在文件里留下标记:

```
<<<<<<< HEAD
官方仓库现在的内容
=======
你的修改
>>>>>>> fix/typo-preface
```

打开文件，保留正确的内容，删掉三行标记，然后:

```
git add <冲突文件>
git rebase --continue
```

如果你在做 merge 而不是 rebase，最后一步用 `git commit`。不确定时先运行 `git status`，它会告诉你当前在 rebase、merge 还是普通状态。冲突太多或 rebase 乱了，可以运行 `git rebase --abort` 回到开始前的状态。

7 Markdown 基础

Markdown 用普通字符表示格式。写大部分内容只需要这些语法：

写法	效果
`# 一级标题`	页面主标题，一个文件通常只有一个
`## 二级标题`	一节
`### 三级标题`	一小节
`**加粗**`	**加粗**
`*斜体*`	斜体
`- 列表项`	无序列表
`1. 列表项`	有序列表
`[文字](https://example.com)`	链接
`` `行内代码` ``	`行内代码`
`> 引用`	引用块
`![说明](图片路径)`	图片
`[^1]`	脚注标记

代码块用三个反引号包起来，并在开头写语言名：

````
```python
def add(a, b):
    return a + b
```
````

表格用竖线分隔：

```
| 方法 | 参数量 | 准确率 |
| --- | --- | --- |
| 全连接 | 235,000 | 98.0% |
| CNN | 61,706 | 99.0% |
```

段落之间空一行。标题、列表、代码块、表格的前后也留空行。少一个空行，Sphinx 可能把两段连在一起。

GitHub 的 Markdown 预览能看标题、列表和表格，但看不懂本项目的 {ref}、{doc}、{note} 和 {literalinclude}。最终效果以本地 `make html` 为准。

8 本项目的 MyST 写法

完整语法在Sphinx Markdown 语法参考。

8.1 文件骨架

新章节参照现有文件的结构：

```
(cnn-basics)=
# 卷积神经网络基础

{doc}`cnn-expedition/index`中我们看到了 CNN 的整体位置。本节讨论卷积层如何提取局部特征。

## 历史背景

...

## 主体内容

### 直观理解

...

### 数学表达

...

### 代码实践

...

## 总结

| 概念 | 实践 | 效果 |
| --- | --- | --- |
| 局部感受野 | 3×3 卷积核 | 只看邻居 |

## 下一步

{doc}`le-net`将把这些层组合成完整的 LeNet。
```

(cnn-basics)= 是标签，必须紧贴在标题上一行。标签在同一本书里不能重复。

8.2 引用其他章节

写法	用途	例子
{ref}	引用标签	{ref}`cnn-basics`
{doc}	引用整篇文档	{doc}`cnn-expedition/index`
{cite}	引用文献	{cite}`vaswani2017attention`

实际写法：

```
回顾{ref}`inductive-bias`中的讨论。

下一节见{doc}`le-net`。

Transformer 的原始论文见{cite}`vaswani2017attention`。
```

{doc} 的路径相对于 source/，不带 .md。{ref} 只写标签名，不带括号。

8.3 提示框

````
```{note}
这是一条补充说明。
```

```{tip}
这是一条建议。
```

```{warning}
这里要提醒读者注意。
```
````

可用的类型有 note、tip、warning、caution、danger、important、hint、attention、error。

8.4 Mermaid 图

项目里的 Mermaid 常用波浪线包起来，避免和外层代码块冲突：

````
~~~{mermaid}
flowchart LR
    A["输入图像"] --> B["卷积层"]
    B --> C["池化层"]
    C --> D["全连接层"]
~~~
````

Mermaid 适合画流程图、结构图和时间线。复杂网络结构用 TikZ，语法在Sphinx Markdown 语法参考。

截图和照片放在 Markdown 文件旁边或 `_static/images/`，用相对路径引用。图片宽度控制在 400 到 600 px，并写清替代文字（alt text）。

8.5 数学公式

行内公式用美元符号包住：

```
卷积输出的大小是 $W_{out} = W_{in} - k + 1$。
```

独立公式用两个美元符号：

```
$$
Y[i,j] = \sum_{u=0}^{k-1}\sum_{v=0}^{k-1} X[i+u,j+v] \cdot K[u,v]
$$
```

行间公式前后留空行。公式里不要用中文标点。

8.6 代码

短代码可以直接写在正文里。长代码放在章节的 `code/` 目录，用 `literalinclude` 引用：

````
```{literalinclude} code/batch-normal.py
:language: python
:linenos:
:caption: 带 BatchNorm 的 CNN
```
````

这样做有两个好处：代码文件可以被单独运行，MyST 也不会把代码里的花括号当成指令。代码注释写清楚维度、参数计算和对应的理论。项目要求比较细，例子：

```python
def forward(self, x):
    # 输入 x: [batch, 1, 28, 28]
    x = self.conv1(x)  # 1→32 通道，3×3 卷积，输出 [batch, 32, 26, 26]
    x = F.relu(x)      # 非线性激活
    x = self.pool(x)   # 2×2 最大池化，输出 [batch, 32, 13, 13]
    return x
```

8.7 表格

普通 Markdown 表格够用。需要多级表头或把指令嵌进表格时，用 `{list-table}`：

````
```{list-table} 全连接与 CNN 对比
:header-rows: 1

* - 模型
  - 参数量
  - 准确率
```
````

```
* - 全连接
  - 235,000
  - 98.0%
* - CNN
  - 61,706
  - 99.0%
```

8.8 文献

所有条目放在 source/references.bib，key 用“作者姓氏 + 年份 + 关键词”：

```
@inproceedings{vaswani2017attention,
  title={Attention is all you need},
  author={Vaswani, Ashish and others},
  booktitle={Advances in Neural Information Processing Systems},
  volume={30},
  year={2017}
}
```

正文用 {cite} 引用。文件末尾加参考文献列表：

```
## 参考文献

```{bibliography}
:filter: docname in docnames
```
```

:filter: docname in docnames 只列出当前页面实际引用过的条目。不要把所有文献都列在每一页。

8.9 目录树

每个章节的 index.md 用隐藏的 {toctree} 定义阅读顺序：

```
```{toctree}
:maxdepth: 2
:hidden:

introduction
cnn-basics
le-net
the-end
```
```

条目是相对路径，不带 `.md`。新增文件后，记得把它加进对应章节的 `index.md`，否则读者在网页导航里找不到。

9 写作要求

项目的写作约定总结如下，完整语法见Sphinx Markdown 语法参考。最常用的几条：

要求	做法
先建立直觉	先用生活类比或几何直觉说明“为什么”，再给公式
用对比帮助理解	用表格比较全连接与 CNN、RNN 与 Transformer
用数字支撑	写清参数量、准确率、时间，数字用千分位
代码带注释	注释写维度、参数计算和对应的理论
建立知识网络	每个主要概念至少有一个 {ref} 或 {doc} 链接
交代历史	提到提出者、年份和原始论文
结尾给下一步	总结本节，再链接到下一节

一个概念出现时，先链接到它的定义或前置章节。比如第一次提到反向传播，就用 {ref} 链接到对应标签，读者可以跳回去补基础。

章节结构按现有文件来：`index.md` 放概览和 toctree，中间放内容文件，最后用 `the-end.md` 收尾。不要单独新建一个没有 toctree 的目录。

术语要统一。同一个概念第一次出现时给出英文对照，后面用同一个中文译名。数字用千分位，百分比保留一位小数。

10 代码贡献

改文字不需要会 Python。只有修改 `code/` 里的示例或新增代码时，才需要运行 Python。

正文里的代码和 `code/` 目录里的脚本是两回事。正文用 `literalinclude` 引用脚本，读者看到的是同一份代码。

新增或修改代码时注意这些：

- 脚本放在对应章节的 `code/` 目录，文件名用能看懂的小写加连字符。
- 代码要能单独运行。需要数据或 GPU 时，在文件开头写清依赖和运行方式。
- 依赖变化时更新 `pyproject.toml` 和 `uv.lock`，在 PR 里说明为什么加。
- 不要提交数据集、模型权重、训练日志、`.pth` 或 `.ckpt` 文件。
- 不要提交 API key、密码、token、个人路径和内网地址。
- 不能运行代码时，在 PR 里写“未运行，缺少 PyTorch/GPU”，不要假装跑过。

终端最常用的命令：

命令	作用
`pwd`	显示当前目录
`ls`	列出文件
`cd <目录>`	进入目录
`cd ..`	回到上一级
`mkdir <目录>`	新建目录
`cp <源> <目标>`	复制文件
`mv <源> <目标>`	移动或重命名
`rm <文件>`	删除文件，删了不能恢复
`cat <文件>`	显示文件内容
`grep -rn "文字" .`	在当前目录递归搜索文字

Python 报错时，先看 traceback 的最后一行。它通常写着错误类型和原因，例如 `ModuleNotFoundError: No module named 'torch'`。把完整错误贴到 Issue 或聊天里，比只说“跑不起来”更容易得到帮助。

11 翻译贡献

中文 Markdown 是唯一的内容源。英文翻译放在 source/locale/en/LC_MESSAGES/ 下，和中文文件路径对应。

改中文后，需要刷新翻译目录：

```
sphinx-build -W --keep-going -b gettext source build/i18n/gettext
sphinx-intl update -p build/i18n/gettext -l en
python scripts/check-i18n.py
```

规则在本地化工作指南，术语在术语表。几条容易犯的错：

- 不要新建一棵英文 Markdown 源树。
- 不要改 {doc}、{ref} 的标签、引用 key、URL、公式、文件名和代码逻辑。
- 不要把中文 msgid 直接复制成 msgstr 来假装翻译完成。
- 翻译完要检查渲染后的 HTML，表格和 TikZ 里的文字可能不会被 gettext 提取。
- 一次只做一个顶层章节，方便英文审校者审阅。
- 不要提交 build/、.pot 或 .mo 文件，只提交 .po 目录。

完整检查：

```
python scripts/check-i18n.py --require-complete --prefix math-fundamentals
bash scripts/build-i18n-site.sh --html-only
```

12 提交前检查

□ git status 里只有你打算提交的文件

□ git diff 里没有误删、个人路径、密钥或无关格式改动

□ make html 能通过，新增的 WARNING 已处理

□ 新增的 {ref} 和 {doc} 都能跳转

□ 代码能运行，或已在 PR 里说明未运行的原因

□ Mermaid/TikZ 能渲染，或已说明缺少依赖

□ 新文件已加入对应章节的 index.md 的 toctree

□ 提交信息说清了改什么

□ PR 描述包含原因和验证方式

□ 翻译改动已运行 check-i18n.py

13 常见问题

现象	原因	处理
`fatal: not a git repository`	终端不在仓库目录	`cd` 到仓库根目录
`remote origin already exists`	已经有同名远程	`git remote set-url origin <地址>`
`Updates were rejected`	远程有你本地没有的提交	`git pull --rebase`，解决冲突后再 push
`CONFLICT (content)`	同一段文字被两边修改	打开文件改掉冲突标记，`git add` 后继续
`make: command not found`	Windows 或环境没装 make	用 `make.bat html`，或改用 WSL
`uv: command not found`	没装 uv	用 `python3 -m venv` 和 `pip install -e .`
`No module named sphinx`	虚拟环境没激活	Linux/macOS 用 `source .venv/bin/activate`，Windows 用 `.venv\Scripts\activate`
Mermaid 图不显示	没装 Chromium 或 mermaid-cli	安装依赖，或先接受图表缺失继续改文字
中文 PDF 显示方框	缺中文字体	安装 Noto Sans CJK，见 build-docs 附录
`{ref}` 找不到	标签拼错或没定义	检查标签拼写和位置，重新构建看警告
`{doc}` 找不到	路径多了 `.md` 或写错层级	路径相对 `source/`，不带扩展名
页面没更新	浏览器缓存或构建目录旧	刷新，必要时 `rm -rf build/html` 重构建
误提交了 `build/`	`git add .` 带进了产物	`git rm -r --cached build`，它已在 `.gitignore` 里
误提交了密钥	文件里有 token 或密码	立刻撤销密钥，通知维护者，仅删文件不够

命令报错时，先判断是哪一步出的问题：

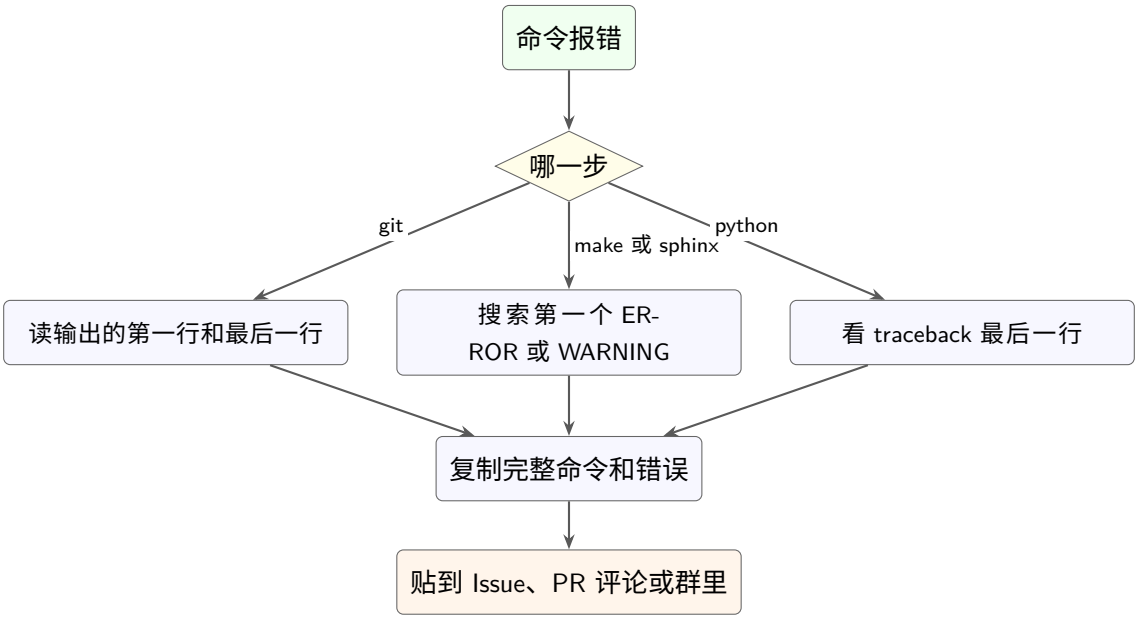


图 18: 报错时的排查顺序

求助时写清四件事：你运行了什么命令、完整错误是什么、操作系统是什么、你已经试过什么。截图可以，但文字更方便别人搜索和复制。

14 术语表

英文	中文	说明
repository	仓库	带历史记录的项目目录
fork	派生副本	复制到你自己账号下的仓库
clone	克隆	把远程仓库下载到本地
branch	分支	一条独立的修改线
commit	提交	一次有说明的修改记录
push	推送	把本地提交上传到远程
pull	拉取	把远程更新下载并合并到本地
remote	远程	另一份仓库的地址
upstream	上游	本项目中指官方仓库
origin	源	本项目中指你的 fork
staging area	暂存区	准备写进下一次提交的改动
merge	合并	把一条分支接回另一条
conflict	冲突	两边改了同一处，需要手动选择
Pull Request	拉取请求	请求官方仓库合并你的分支
Issue	议题	讨论问题或提出修改建议
CI	持续集成	自动构建和检查，本项目用 GitHub Actions

15 接下来做什么

从一个具体的文件开始。打开 `source/preface.md`，读一遍，找一句可以写得更清楚的话，或者找一个错别字。按第 4 节在网页上改掉它，走一次完整的 PR 流程。

第二次可以选一个规模不大的 Issue，或者给某一节补一张 Mermaid 图。改之前先读相邻章节和Sphinx Markdown 语法参考，确认术语和结构一致。提交 PR 时写清你运行过的命令和看到的页面，审阅者会更快理解你的改动。

16 相关链接

- 项目仓库
- 项目 README
- 在线文档
- Issue 列表
- Pull requests
- Actions 构建记录
- Sphinx Markdown 语法参考
- 本地构建说明
- 本地化工作指南
- 术语表